AthenaAI

Modular AI Runtime — Phase-by-Phase Study Notes

---

15 phases (P0–P14) · Python 3.12+ · uv monorepo · FastAPI · asyncpg · pgvector · Redis · OpenTelemetry

Architecture layers:

1. **AthenaRuntime** — central engine (model selection, RAG, memory, tools, agents, resilience, observability)
2. **AthenaGateway** — FastAPI public surface (auth, rate limiting, SSE streaming)
3. **AthenaEval** — evaluation harness (correctness, latency, cost, retrieval quality)

Single entry point principle:

```
response = await runtime.execute(request)
```

Caller never knows if a model was selected, docs retrieved, tools called, retries happened, or traces recorded.

Operating Loop (Every Phase)

Every phase follows this exact sequence — no skipping:

Step	Action	Command / Check
1. PLAN	List files to create + why	—
2. IMPLEMENT	Create files, smallest dep first	—
3. LINT/TYPE	Run ruff + mypy	uv run mypy src/ && uv run ruff check src/
4. TEST	Run gate tests	uv run pytest tests/<module>/ -v
5. GATE	Verify gate condition	All tests green
6. COMMIT	Git commit	git add -A && git commit -m 'feat(scope): ...'
7. REPORT	Print phase complete	—
8. LOOP	Start next phase	—

Non-negotiable rules enforced every file:

Protocol over ABC — ALL interfaces in core/protocols.py as typing.Protocol

No raw dict between components — use @dataclass(frozen=True)

No isinstance chains — use match statement on tagged types

No threading.Thread for I/O — async def + await everywhere

No eval() in CalculatorTool — use restricted AST walk

No silent truncation — raise ContextOverflowError

No recursive DFS in AgentPlanner — explicit stack only

No TODO stubs — implement fully or skip the file

P0 — Core Types + Project Scaffold

Goal: Project installs, types resolve, mock runtime executes.

Files Created

File	Purpose
pyproject.toml	uv project root — all 20+ deps declared
src/athenai/__init__.py	__version__ = '0.1.0'
core/exceptions.py	8 custom exceptions: ContextOverflowError, ModelUnavailableError, ToolDeniedError, ToolT
core/types.py	6 frozen dataclasses: Message, TokenUsage, AIRequest, AIResponse, TraceSpan, RoutingC
core/protocols.py	7 Protocols: Model, StreamingModel, MemoryStore, Retriever, Tool, CacheBackend, Embedd
core/config.py	AthenaConfig(BaseSettings) — env-driven, ATHENA_ prefix
core/lifecycle.py	@asynccontextmanager lifespan() — component registry as dict[str, Any]

Key Design Decisions

WHY Protocol over ABC: Structural subtyping — MockModel satisfies Model without inheriting from it. Decouples interface from implementation, eliminates circular imports, no MRO overhead on hot path.

WHY frozen=True dataclasses: Requests flow through routing, context, model, observability pipeline. Immutability prevents mid-pipeline mutation and makes types safe as dict keys for caching.

WHY asynccontextmanager for lifecycle: Single function owns startup/shutdown order explicitly. No scattered try/finally blocks. Cleanup guaranteed even if startup fails mid-way.

WHY ContextOverflowError (not silent truncation): Silent drops cause hallucination with no observable cause. Hard ceiling forces caller to make an explicit decision about what to sacrifice.

Gate Tests (tests/unit/test_types.py)

Gate: Instantiate every @dataclass(frozen=True) — no exception. Assert frozen: assigning to field raises FrozenInstanceError. AIRequest with empty messages list raises ValueError. Import all Protocols — no ImportError. AthenaConfig loads from environment variables correctly.

```
uv run pytest tests/unit/test_types.py -v
```

P1 — Model Layer

Goal: Mock and cloud model adapters working, registry config-driven.

Files Created

File	Key Implementation
models/base.py	ModelRequest, ModelResponse frozen dataclasses
models/mock.py	MockModel: echo prompt back, deterministic token counts, stream() yields chars with 10ms sleep
models/cloud.py	CloudModel: Anthropic API via httpx.AsyncClient. 429 → RateLimitError, 5xx → ModelUnavailableEr
models/local.py	LocalModel: Ollama /api/generate via httpx
models/registry.py	ModelRegistry: config-driven, get(role), list_roles(), health_check(role)

Key Design Decisions

WHY MockModel: Deterministic, no API key needed. Tests run in CI without credentials. Echo-back makes assertions trivial — expected output is derivable from input.

WHY httpx over Anthropic SDK: httpx.AsyncClient is a standard async HTTP client with predictable timeout/retry behaviour. SDK hides HTTP details that matter for resilience (status codes, connection reuse, timeout propagation). respx can mock it cleanly in tests.

Gate Tests (tests/unit/test_models.py)

MockModel.generate() returns ModelResponse with non-empty content. MockModel.stream() yields at least 3 tokens. ModelRegistry.get('fast') returns configured model. ModelRegistry.get('nonexistent') raises KeyError. CloudModel + mock 401 → ModelUnavailableError. CloudModel + mock 429 → RateLimitError. Registry round-trip: register mock as 'fast' + 'reasoning', get both.

```
uv run pytest tests/unit/test_models.py -v
```

P2 — Resilience

Goal: Retry, timeout, circuit breaker, rate limiter all working and testable.

Files Created

File	Key Implementation
resilience/retry.py	RetryPolicy(frozen=True): max_attempts, base_delay_s, max_delay_s, jitter: bool. async with_re
resilience/timeout.py	async with_timeout(fn, seconds) — asyncio.wait_for wrapper, raises ToolTimeoutError on asynci
resilience/circuit_breaker.py	CircuitBreaker: asyncio.Lock for CAS state transitions. States: CLOSED/OPEN/HALF_OPEN. fai
resilience/rate_limiter.py	TokenBucketRateLimiter: asyncio.Lock guards token count + last_refill atomically. async acquire

Key Design Decisions

WHY asyncio.Lock for CAS in CircuitBreaker: Without a lock, two concurrent tasks could both read CLOSED, both count failures, and both attempt the CLOSED→OPEN transition independently. The lock makes read-modify-write atomic, so exactly one transition fires regardless of concurrency level.

WHY one Lock covers both fields in TokenBucketRateLimiter: token_count and last_refill must be updated together — if another coroutine reads between the refill and the decrement, it sees an inconsistent state. One lock covers both atomically.

WHY full jitter on retry backoff: Thundering herd — without jitter, all N failing clients retry at the same instant after backoff, causing correlated load spikes. Full jitter spreads retries randomly across [0, delay] so the aggregate load stays flat.

Gate Tests

Circuit breaker: 2 failures → OPEN. Sleep cooldown → HALF_OPEN. 1 success → CLOSED. 20 concurrent tasks all calling record_failure() → exactly one CLOSED→OPEN transition (test with counter).

Rate limiter: capacity=5: 5 acquire() succeed, 6th raises RateLimitError. Drain, sleep 0.1s, 5 more succeed (refilled). 10 concurrent tasks with capacity=3 → exactly 3 succeed.

```
uv run pytest tests/unit/test_circuit_breaker.py tests/unit/test_rate_limiter.py -v
```

P3 — Model Router

Goal: Cost/latency/quality-aware router selecting models by RoutingContext.

Files Created

File	Key Implementation
routing/policies.py	RoutingPolicy(frozen=True): quality_weight, cost_weight, latency_weight, max_cost_usd, max_laten
routing/scorer.py	ModelScorer: weighted score = quality*w_q + (1/cost)*w_c + (1/latency)*w_l from ModelMetadata
routing/router.py	ModelRouter.select(context, policy) → RoutingDecision(frozen=True). Classifies LOW/MEDIUM/HIG

Key Design Decisions

WHY weighted scoring over hard cutoff routing: Hard cutoffs (if latency < X: use fast model) require manual tuning per deployment. Weighted scoring lets the routing policy express tradeoffs declaratively — cost-sensitive configs increase cost_weight, quality-critical configs increase quality_weight — without changing routing code.

Complexity classification: token_estimate < 500 = LOW, < 2000 = MEDIUM, else HIGH. LOW routes to 'fast' model, HIGH routes to 'reasoning' model regardless of other weights.

Gate Tests (tests/unit/test_router.py)

'Summarise this sentence' → LOW → fast model. 'Design a distributed payment system' → HIGH → reasoning. 'Translate to French' (quality low, cost high) → fast model. All models circuit-OPEN → ModelUnavailableError. estimated_cost_usd positive and < max_cost_usd. Custom weights → scorer picks different model.

```
uv run pytest tests/unit/test_router.py -v
```

P4 — Context Engine + Token Budget

Goal: Token-budgeted context assembly, parallel memory+RAG retrieval.

Files Created

File	Key Implementation
context/budget.py	TokenBudgetManager: allocations dict[str, int]. allocate(key, tokens) returns actually allocated, raises
context/ranking.py	RelevanceRanker: cosine similarity via numpy between query embedding and chunk embeddings. R
context/packing.py	ContextPacker: inserts in priority order system > conversation > memory > rag > tools. Truncates low
context/engine.py	ContextEngine.build() — asyncio.gather(memory, rag) parallel retrieval. Calls packer. Returns BuiltC

Key Design Decisions

WHY asyncio.gather for memory+RAG: Memory retrieval (DB query) and RAG retrieval (vector search) have no data dependency — they can run in parallel. If memory takes 100ms and RAG takes 200ms, sequential takes 300ms, parallel takes 200ms. At scale this matters significantly.

WHY hard token ceiling (not soft): Silent truncation hides bugs in token estimation and causes non-deterministic model behaviour. A hard ceiling (ContextOverflowError) forces the caller to decide what to drop — system prompt? RAG chunks? conversation history?

Truncation priority (lowest first): tools < rag < memory < conversation < system. System prompt is never truncated — it defines model behaviour. RAG chunks are cheapest to drop.

Gate Tests (tests/unit/test_context_budget.py)

4 buckets within total → no exception, sum equals total. Allocate past ceiling → ContextOverflowError. Packer: 3 buckets, last overflows → truncate rag bucket, not system/conversation. ContextEngine.build() → BuiltContext has all expected fields, total <= ceiling. Parallel test: memory 100ms + RAG 200ms → build() completes in <250ms (not 300ms).

```
uv run pytest tests/unit/test_context_budget.py -v
```

P5 — Memory

Goal: Four memory layers, conversation persists across calls.

Layer	Backend	Key Feature
WorkingMemory (base.py)	In-process	MemoryEntry(frozen=True), MemoryType enum
AgentState (working.py)	In-process mutable	Holds task, steps, tool_results, status
ConversationMemory (conversation.py)	asyncpg (PostgreSQL)	add_message(), get_recent(n), clear(session_id)
SummaryMemory (summary.py)	PostgreSQL + model call	Compresses older msgs when > 20 raw. Returns summary + latest 5 ra
SemanticMemory (semantic.py)	pgvector	store(fact), search(query_embedding, k) with cosine <=> operator

WHY PostgreSQL for ConversationMemory (not Redis): Conversation history needs durability across server restarts. Redis is ephemeral by default. SemanticMemory needs pgvector which is PostgreSQL-native. Using one DB for both simplifies ops.

Gate: uv run pytest tests/integration/test_memory.py -v (needs test DB)

P6 — RAG Pipeline

Goal: Document ingestion + retrieval end-to-end.

File	Key Detail
rag/parser.py	PDF via pypdf, TXT/MD as-is. Returns ParsedDocument(text, source, metadata).
rag/chunker.py	SlidingWindowChunker: chunk_size + overlap in tokens. Overlap prevents split-sentence retrieval m
rag/embedder.py	CloudEmbedder: Anthropic embeddings API via httpx. embed(texts: list[str]) → list[list[float]]. Batch-a
rag/retriever.py	PgVectorRetriever: cosine search on chunks table. search(query_embedding, k, metadata_filter).
rag/reranker.py	PassthroughReranker (no-op) + CrossEncoderReranker stub. Reranker optional/swappable.
rag/loader.py	DocumentLoader.ingest(): parse → chunk → embed → store. Idempotent on document_id.

Gate: uv run pytest tests/unit/test_chunker.py tests/integration/test_rag_pipeline.py -v

---

P7 — Tool System

Goal: Tool registry, schema validation, permission check, three tools working.

Validation order: JSON Schema first, then permission check.
WHY schema before permission: Schema validation is stateless/cheap. Permission check may require a DB lookup. Fail fast on the cheapest check.

CalculatorTool: Uses ast module — walks AST nodes, allows only Add/Sub/Mul/Div/Pow/BinOp/Num. Any other node (Import, Call, Attribute) → ToolDeniedError. NEVER eval().
SQLTool: Read-only asyncpg pool. Rejects non-SELECT. Parameterised queries only.
HTTPTool: Allowlist-only domains. httpx.AsyncClient 10s timeout.

Gate: uv run pytest tests/unit/test_tool_validator.py tests/integration/test_tool_execution.py -v

---

P8 — Agent Runtime

Goal: Stateful agent with state machine, iterative DAG planner, tool loop.

Component	Key Detail
AgentStatus (state.py)	Enum: CREATED/PLANNING/WAITING_TOOL/EXECUTING_TOOL/COMPLETED/FAILED/CANCE
AgentPlanner (planner.py)	plan(task) → ExecutionDAG. topological_sort() uses iterative DFS with explicit stack — no recursion
AgentExecutor (executor.py)	match on AgentStatus to dispatch. max_iterations=10 hard cap. asyncio.gather for parallel independ
Agent (agent.py)	run(task, context, tools) → AgentResult. Composes planner + executor + working memory. Writes A

WHY iterative DFS: Python default recursion limit is 1000. A large DAG with 100+ tool nodes would hit it. Explicit stack is O(n) space, no stack overflow risk, and easier to add cycle detection.

WHY state machine over while-True loop: Explicit state transitions make illegal paths compile-checkable. A COMPLETED agent cannot transition to PLANNING — the code enforces it, not just the comments.

Gate: uv run pytest tests/unit/test_agent_state.py tests/integration/test_agent_run.py -v

P9 — API Gateway + Full Pipeline

Goal: FastAPI endpoints wired to AthenaRuntime, streaming working.

Endpoint	Method	Description
/v1/chat	POST	Sync chat → AIResponse JSON
/v1/chat/stream	POST	SSE streaming — at least 3 events
/v1/agents/run	POST	Agent task execution
/v1/documents	POST/GET	Document ingestion (202 + job_id) / list
/v1/runs/{run_id}	GET	Agent run status (404 if unknown)
/health	GET	{'status': 'ok'}
/ready	GET	200 or 503 if DB unreachable
/metrics	GET	Prometheus text format

BoundedExecutor: asyncio.Semaphore(max_concurrent_model_calls) wraps ALL model calls. Prevents unbounded gather() from spawning unlimited concurrent API calls.

Gate: uv run pytest tests/integration/test_chat_endpoint.py -v

---

P10 — Observability

Prometheus metrics: requests_total{model,status}, model_latency_seconds{model} (histogram, buckets: 0.1/0.5/1/2/5/10s), tokens_total{model,type}, cache_hits_total, tool_calls_total{tool,status}, rag_chunks_retrieved_total, agent_iterations_total

Trace structure: Every request gets trace_id. Spans: gateway → routing → context_build → model_call. Each span has name, start_ms, end_ms, attributes dict. Stored in DB for replay.

WHY structured trace vs log tailing: A single AI request involves routing decisions, parallel retrieval, tool dispatch — filtering thousands of log lines for one request_id is impractical. A structured span tree lets you replay the exact execution path instantly.

P11 — Failure Scenarios

Model timeout: Primary sleeps 30s, timeout=2s → completes in <3s with fallback. Trace shows model_timeout, circuit_open, fallback_model_used.
Circuit recovery: 5 failures → OPEN → cooldown → HALF_OPEN → probe → CLOSED.
Context overflow: 120% ceiling → ContextOverflowError → 400 (not 500). RAG trimmed first.
Tool denied: User without 'sql' permission + SQL tool → ToolDeniedError → 403. Metric +1.

Gate: uv run pytest tests/failure/ -v (no new source files — exercises resilience layer)

P12 — Benchmarks

bench_context_engine.py: Sequential vs asyncio.gather retrieval, 100 iterations, mean + p95.
bench_concurrent_requests.py: 10/100/1000 concurrent execute() calls, p50/p95/p99, req/s, error rate.

Gate criteria: Parallel retrieval >= 30% faster than sequential. p99 MockModel at 1000 concurrent < 500ms. Zero errors at 10 concurrent. Error rate < 1% at 1000.

P13 — Evaluation Harness

EvalDataset: Loads JSONL with question/expected_answer/documents. Validates on load.
EvalMetrics: correctness (exact + token overlap), latency_ms, cost_usd, hallucination_score.
EvalRunner: async batch execute against runtime, concurrency=5. EvalReport.to_markdown() → comparison table.

P14 — README + Docker

docker-compose.yml services: athena-api (:8000), postgres (:5432 + pgvector), redis (:6379), prometheus (:9090), grafana (:3000).
Dockerfile: Multi-stage — build stage installs with uv, runtime stage is slim Python 3.12.
Gate: git clone → make install → make up → curl POST /v1/chat → 200 with content, trace_id, model, usage.

Interview Cheat Sheet — Design Decisions

Pattern	Where Used	Why (2-line answer)
Protocol over ABC	core/protocols.py	Structural subtyping — MockModel satisfies Model without inheriting. Decouples interfac
asyncio.gather	context/engine.py	Memory + RAG have no data dependency. Parallel cuts latency from 300ms to 200ms at
@dataclass(frozen=True)	All value types	Requests flow through entire pipeline. Immutability prevents mid-pipeline mutation; types
Iterative DFS	agents/planner.py	Python recursion limit is 1000. Large DAGs hit it. Explicit stack is O(n) space, no stack ov
Token budget hard ceiling	context/budget.py	Silent truncation causes hallucination with no observable cause. Hard ceiling forces expli
CAS via asyncio.Lock	resilience/circuit_breaker.py	TOCTOU — two concurrent tasks could both read CLOSED and both fire OPEN transitio
Semaphore on model calls	runtime/executor.py	Unbounded asyncio.gather() with 1000 concurrent requests → 1000 API calls. Semapho
Schema before permission check	tools/validator.py	Schema validation is stateless/cheap. Permission check may need DB lookup. Fail fast o
Single execute() entry point	runtime/runtime.py	Caller decoupled from all runtime internals. Cache, retry, fallback transparent. Testable in
PassthroughReranker	rag/reranker.py	Reranking is expensive. Default no-op keeps latency low. Cross-encoder swappable with
ConversationMemory in PostgreSQL	memory/conversation.py	Durability across restarts. Redis is ephemeral. pgvector already requires PostgreSQL —
structlog bound context	observability/logger.py	Every log line carries trace_id, request_id, user_id automatically. No manual log.info('trac

---

Request Lifecycle (Request → AIResponse)

1. **RequestParser** — validate, assign request_id, trace_id
2. **PolicyCheck** — content policy, user permissions
3. **IntentAnalysis** — task classification, complexity estimate
4. **ModelRouter** — select model(s), estimate cost + latency
5. **ContextEngine** — asyncio.gather(memory, rag) in parallel
6. **AgentRuntime** — plan → tool call loop → final answer
7. **LLM Request** — via selected Model adapter
8. **Observability** — record trace, tokens, cost, latency
9. **AIResponse** — content, model, usage, trace_id